

A geometric and Bayesian formulation of autoregressive language models

Akshay Mishra

June 8, 2026

Abstract

The mechanics of autoregressive large language models (LLMs) are formalized by re-framing the forward pass and pre-training objectives through the lens of Bayesian inference, high-dimensional geometry, and statistical mechanics. The conditional probability distribution generated by the Transformer architecture is shown to be explicitly decomposable into an intrinsic token prior and a contextual likelihood. These probabilistic components are mapped directly onto the structural blocks of the network—specifically multi-head attention (MHA) and feed-forward networks (FFN). It is demonstrated that the final output token distribution is modulated by the statistical variance of the latent residual stream, which acts as a dynamic inverse temperature. Finally, pre-training is formalized as a self-organizing geometric optimization by which these components are resolved into a stable semantic coordinate system.

1 Introduction and the Bayesian framing

An autoregressive large language model operates over a finite vocabulary V , framing sequence prediction as originally formalized by Shannon [11] and implemented via neural networks by Bengio et al. [2] and Mikolov et al. [19]. Given a context sequence of n tokens, $C = (t_1, t_2, \dots, t_n)$, where $t_i \in V$, a discrete conditional probability distribution over the vocabulary is defined for the next token $w \in V$:

$$P(w \mid C) \tag{1}$$

While this distribution is directly evaluated by modern deep learning architectures using discriminative causal language modeling [3], the formulation can be inverted using Bayes' theorem to expose the underlying generative dynamics [9]:

$$P(w \mid C) = \frac{P(C \mid w) \cdot P(w)}{P(C)} \tag{2}$$

Since the denominator $P(C) = \sum_{v \in V} P(C \mid v)P(v)$ represents the marginal likelihood of the context sequence, it serves as a constant normalization factor across all candidate tokens at step $n + 1$. The relative probability mass assigned to any token w is therefore determined strictly by the numerator:

$$P(w \mid C) \propto P(C \mid w) \cdot P(w) \tag{3}$$

where:

1. $P(w)$ denotes the **unconditional prior probability** of token w occurring in the language, independent of local context.

2. $P(C \mid w)$ represents the **contextual likelihood**, measuring the alignment of the historical sequence C with the trailing token w .

Intuitively, we can think of the model’s next-token prediction as an inference problem where the model weighs the default statistical frequency of a word (the prior) against how well that word explains or aligns with the observed prefix sequence (the likelihood). For instance, common words like “the” or “of” start with a high prior probability, but a highly specific context (e.g., “The capital of France is”) drastically shifts the likelihood in favor of specific target tokens like “Paris”. Bayes’ theorem provides a clean mathematical decomposition of this tension between global frequency and local context.

2 Mathematical decomposition of likelihood $P(C \mid w)$

In a Transformer architecture [1], the context sequence C is transformed into a sequence of continuous vectors. The input embedding matrix for the context is denoted as $X \in \mathbb{R}^{n \times d}$, where d is the model dimension. The final hidden state at the terminal position n after L layers is denoted as $h_n^{(L)} \in \mathbb{R}^d$.

The unnormalized log-probabilities (logits) z_w for each token w are computed by projecting $h_n^{(L)}$ onto an unembedding matrix $U \in \mathbb{R}^{|V| \times d}$, such that $z_w = h_n^{(L)} \cdot u_w$, where $u_w \in \mathbb{R}^d$ is the unembedding vector of the target token [4].

The log-likelihood $\log P(C \mid w)$ is shaped by the interaction of two foundational architectural components within each layer l : multi-head attention (MHA) and the position-wise feed-forward network (FFN). Under this perspective, the Transformer’s layers act as a series of belief-updating steps. Instead of computing the likelihood in a single step, the network progressively refines its representation of the context, letting different attention heads and feed-forward sub-layers query, extract, and integrate different aspects of the context. Each component’s output is projected back into a shared vector space—the residual stream—where their contributions are combined.

2.1 Multi-head attention as contextual routing

The multi-head attention block at layer l and position n computes a dynamically weighted combination of all preceding token states [1]. The number of attention heads is denoted by H , and the head dimension by d_k . For a single attention head $g \in \{1, \dots, H\}$, the query, key, and value projection matrices are denoted by $W_Q^{(l,g)}, W_K^{(l,g)}, W_V^{(l,g)} \in \mathbb{R}^{d \times d_k}$, respectively. Mathematically, these matrices serve as linear feature extractors that project the general residual representation $h^{(l-1)}$ into specialized spaces:

1. The query projection $W_Q^{(l,g)}$ maps the current token state to a query vector $q_n^{(l,g)} = h_n^{(l-1)} W_Q^{(l,g)}$, representing the specific contextual information being requested at the terminal position n .
2. The key projection $W_K^{(l,g)}$ maps historical token states to key vectors $k_i^{(l,g)} = h_i^{(l-1)} W_K^{(l,g)}$, representing the indexable features or keys offered by each preceding token i .
3. The value projection $W_V^{(l,g)}$ maps historical token states to value vectors $v_i^{(l,g)} = h_i^{(l-1)} W_V^{(l,g)}$, representing the actual semantic content or updates to be extracted if a match occurs.

The dynamic attention weights $\alpha_{n,i}^{(l,g)}$ represent the relevance or routing strength of each historical position i relative to the current position n . These weights are computed by taking the dot product between the query $q_n^{(l,g)}$ and each key $k_i^{(l,g)}$, scaled by $\sqrt{d_k}$ to prevent gradient vanishing, and normalized across all positions via the softmax function:

$$\alpha_{n,i}^{(l,g)} = \frac{\exp\left(\frac{q_n^{(l,g)} \cdot k_i^{(l,g)}}{\sqrt{d_k}}\right)}{\sum_{j=1}^n \exp\left(\frac{q_n^{(l,g)} \cdot k_j^{(l,g)}}{\sqrt{d_k}}\right)} \quad (4)$$

Crucially, the attention weights $\alpha_{n,i}^{(l,g)}$ themselves do not incorporate the value vectors; rather, they serve as a routing distribution that acts on the value vectors. The output of head g is computed as the weighted sum of these value vectors: $a_{n,g}^{(l)} = \sum_{i=1}^n \alpha_{n,i}^{(l,g)} v_i^{(l,g)}$.

The concatenated outputs of all heads are projected back into the residual stream via the global output matrix $W_O^{(l)} \in \mathbb{R}^{d \times d}$. By partitioning $W_O^{(l)} = [(W_{O,1}^{(l)})^T \dots (W_{O,H}^{(l)})^T]^T$, where each head's output projection slice is represented as $W_{O,g}^{(l)} \in \mathbb{R}^{d_k \times d}$, the global attention output is $a_n^{(l)} = \sum_{g=1}^H a_{n,g}^{(l)} W_{O,g}^{(l)}$. Projecting this output onto the target token unembedding vector u_w yields the total attention contribution to the log-likelihood:

$$\log P(C \mid w)_{\text{MHA}} \sim \sum_{g=1}^H \sum_{i=1}^n \alpha_{n,i}^{(l,g)} \left(v_i^{(l,g)} W_{O,g}^{(l)} u_w \right) \quad (5)$$

By expanding $v_i^{(l,g)} = h_i^{(l-1)} W_V^{(l,g)}$, this contribution is expressed in its standard bilinear form, building upon bilinear attention scoring techniques originally developed for sequence alignment [14]:

$$\log P(C \mid w)_{\text{MHA}} \sim \sum_{g=1}^H \sum_{i=1}^n \alpha_{n,i}^{(l,g)} \left(h_i^{(l-1)} W_V^{(l,g)} W_{O,g}^{(l)} u_w \right) \quad (6)$$

Two distinct mechanisms are thereby isolated: contextual routing is determined by $\alpha_{n,i}^{(l,g)}$, whereas semantic alignment between the historical features of token i and target token w is mediated by the bilinear inner product. Intuitively, the query-key interaction $\alpha_{n,i}^{(l,g)}$ acts as a dynamic routing mechanism that scales with the relevance of historical positions i to the query at position n . Upon connection, the model retrieves the contextualized value vector of token i , transforms it through the output projection, and aligns it with the candidate target token w , rather than simply replicating the token i itself. Thus, the attention head computes how much a specific historical event (token i) increases or decreases the likelihood of the next token w based on their semantic compatibility.

2.2 Feed-forward networks as concept memory retrieval

Following the attention block, the updated hidden state $x_n^{(l)} = h_n^{(l-1)} + a_n^{(l)}$ is processed by the position-wise feed-forward network (FFN). The FFN is modeled as a key-value associative memory [5], extending the classical key-value and end-to-end memory network paradigms [20, 15]. The parameters are represented by $W_1^{(l)} \in \mathbb{R}^{d \times d_{ff}}$ (denoting intermediate keys $\{k_m^{(l)}\}_{m=1}^{d_{ff}}$) and $W_2^{(l)} \in \mathbb{R}^{d_{ff} \times d}$ (denoting intermediate values $\{v_m^{(l)}\}_{m=1}^{d_{ff}}$).

Upon application of a non-linear activation function σ (e.g., GELU [6]), the operation at the terminal position is formulated as:

$$\text{FFN}(x_n^{(l)}) = \sum_{m=1}^{d_{ff}} \sigma \left(x_n^{(l)} \cdot k_m^{(l)} + b_{1,m}^{(l)} \right) v_m^{(l)} \quad (7)$$

Projection of this operation onto the target token unembedding vector u_w yields the FFN likelihood formulation:

$$\log P(C \mid w)_{\text{FFN}} \sim \sum_{m=1}^{d_{\text{ff}}} \sigma \left(x_n^{(l)} \cdot k_m^{(l)} + b_{1,m}^{(l)} \right) \cdot (v_m^{(l)} \cdot u_w) \quad (8)$$

The activation term $\sigma \left(x_n^{(l)} \cdot k_m^{(l)} + b_{1,m}^{(l)} \right)$ scales the degree of satisfaction of the m -th latent conceptual rule by the synthesized context, whereas the term $(v_m^{(l)} \cdot u_w)$ represents the semantic correlation between that rule and the vocabulary token w . To build intuition, we can view the FFN as a database of textual patterns or “rules”. The intermediate keys $k_m^{(l)}$ represent key patterns in the language (e.g., “subject is plural” or “context is about computing”). The activation function σ determines if the current hidden state matches this pattern. If it does, the corresponding value vector $v_m^{(l)}$ is retrieved and added to the residual stream. This value vector contains semantic updates that promote or demote specific target tokens w through the dot product $(v_m^{(l)} \cdot u_w)$. For instance, if a rule detecting “context is about computing” is triggered, its value vector will have large positive dot products with target tokens like “server”, “compiler”, or “database”.

3 Additive integration in the residual stream

Before analyzing the individual geometric components of the prior and the final logit mechanics, we must examine how the contributions of the attention and feed-forward blocks accumulate. In the Transformer architecture, the hidden state at the terminal position n is updated via a residual connection after each sub-layer. The final hidden state $h_n^{(L)}$ after L layers can be expanded as the sum of the initial embedding state and the outputs of all attention and feed-forward blocks across all layers:

$$h_n^{(L)} = h_n^{(0)} + \sum_{l=1}^L \left(a_n^{(l)} + f_n^{(l)} \right) \quad (9)$$

where $h_n^{(0)} \in \mathbb{R}^d$ represents the initial input representation (the sum of the token embedding e_{t_n} and positional embedding p_n at position n), $a_n^{(l)} \in \mathbb{R}^d$ is the output of the multi-head attention block at layer l , and $f_n^{(l)} \in \mathbb{R}^d$ is the output of the feed-forward network at layer l .

When the final logit z_w for a candidate token w is computed via the dot product with the unembedding vector u_w , this additive structure is preserved:

$$z_w = h_n^{(L)} \cdot u_w = \left(h_n^{(0)} \cdot u_w \right) + \sum_{l=1}^L \left(a_n^{(l)} \cdot u_w \right) + \sum_{l=1}^L \left(f_n^{(l)} \cdot u_w \right) \quad (10)$$

To relate this additive expansion to the Bayesian formulation, we map the logits to the posterior log-probabilities via the Softmax normalization. The log posterior probability is given by:

$$\log P(w \mid C) = \log P(w) + \log P(C \mid w) - \log P(C) \quad (11)$$

By equating the log-posterior to the normalized logit $\log P(w \mid C) = z_w - \log Z(C)$, where $Z(C) = \sum_{v \in V} \exp(z_v)$ is the context-dependent partition function, the unnormalized logit corresponds directly to the unnormalized joint log-probability up to a constant offset:

$$z_w = \log P(w) + \log P(C \mid w) + (\log Z(C) - \log P(C)) \quad (12)$$

This mapping allows us to identify the statistical counterparts of the residual stream components under a robust generalized additive framework:

1. The initial state projection $(h_n^{(0)} \cdot u_w) = (e_{t_n} \cdot u_w) + (p_n \cdot u_w)$ serves as a local transition prior (capturing bigram log-odds and positional bias relative to the preceding token t_n). This baseline is further modulated by the static, context-independent parameters (unembedding biases and norms) that encode the global unconditional prior $\log P(w)$, as analyzed in Section 4.
2. The sub-layer updates $(a_n^{(l)} \cdot u_w)$ and $(f_n^{(l)} \cdot u_w)$ act as sequential feature contributions. Rather than assuming that the Transformer layers compute independent likelihood updates—which would violate their hierarchical dependencies—this additive summation is modeled as a Generalized Additive Model (GAM) that sequentially refines the unnormalized log-likelihood:

$$\log P(C | w) + \text{constant}_C \approx \sum_{l=1}^L (a_n^{(l)} \cdot u_w + f_n^{(l)} \cdot u_w) \quad (13)$$

Thus, the residual stream serves as an additive log-probability accumulator. Instead of calculating the posterior in a single step, the Transformer functions as a sequential Bayesian estimator: each layer queries the context (via MHA) and applies conceptual rules (via FFN) to add updates to the running log-prior, culminating in the final logit. This additive formalization of the residual stream aligns with its visualization as a shared communication channel through which individual sub-layers write and read semantic features [16], and underpins the projection of intermediate activations onto the unembedding vocabulary space to analyze layer-wise updates, a technique known as the Logit Lens [17].

4 Geometric formulation of the prior $P(w)$ and logit mechanics

The exact computation of the unconditional prior $P(w)$ requires marginalization over the space of all possible context sequences $\mathcal{C}$:

$$P(w) = \sum_{C \in \mathcal{C}} P(w | C) P(C) \quad (14)$$

This expectation is statically approximated within the parameters of the Transformer via two geometric mechanisms. Because calculating the true marginal probability of a word across all possible contexts is computationally intractable, the network stores a static approximation of word frequency directly within its weights. This prior acts as a starting bias: even before reading a sentence, some words (like “the”) are vastly more likely to occur than others. The network encodes this frequency prior in two distinct geometric ways depending on the architecture:

4.1 Unembedding bias (b_U)

In architectures featuring an explicit unembedding bias $b_U \in \mathbb{R}^{|V|}$, the logit calculation is defined as $z_w = (h_n^{(L)} \cdot u_w) + b_{U,w}$. Under an uninformative context where $h_n \rightarrow \mathbf{0}$, the logit collapses directly to the scalar bias component, yielding:

$$P(w) \approx \frac{\exp(b_{U,w})}{\sum_{v \in V} \exp(b_{U,v})} \quad (15)$$

The unembedding bias $b_{U,w}$ functions as a direct lookup table of global token frequencies. When the context is empty or completely uninformative, the network output h_n vanishes, and the softmax probabilities default entirely to these bias values. This is equivalent to predicting the next word based purely on its raw frequency in the training corpus.

4.2 Embedding norms ($\|u_w\|$)

In bias-free architectures, the prior is encoded in the Euclidean magnitude of the token vector, $\|u_w\|$. Geometric decomposition of the final inner product yields:

$$z_w = \|h_n^{(L)}\| \|u_w\| \cos(\theta_w) \quad (16)$$

where θ_w is the angle between the final context state and the token vector. Longer vector magnitudes $\|u_w\|$ are developed during optimization by tokens with high baseline frequencies in the training corpus, thereby amplifying the baseline logit irrespective of the contextual direction $\cos(\theta_w)$ [7]. In the absence of a bias term, the network exploits the magnitude (Euclidean norm) of the token vectors themselves. Because the dot product $h_n \cdot u_w$ depends on the length of u_w , a token with a very long unembedding vector will naturally receive a higher logit, even if the context vector h_n is only weakly aligned with it. During training, the optimization process stretches the vectors of common words and shrinks those of rare words, embedding the prior probability distribution directly into the scale of the vector space.

5 Statistical mechanics of the latent stream

The remaining term in the geometric logit decomposition is the norm of the final hidden state, $\|h_n^{(L)}\|$. This component serves as a dynamic, context-dependent entropy controller.

5.1 Activation variance as inverse temperature

The final hidden state $h_n^{(L)}$ is represented as a row vector $h_n = [x_1, x_2, \dots, x_d] \in \mathbb{R}^d$. If the individual vector components x_i are treated as samples from an internal distribution with a mean driven to zero by layer normalization ($\mu_x \approx 0$) [8], the variance across components is defined as:

$$\sigma^2 = \frac{1}{d} \sum_{i=1}^d (x_i - \mu_x)^2 \approx \frac{1}{d} \sum_{i=1}^d x_i^2 \quad (17)$$

Since the squared Euclidean norm is given by $\|h_n\|^2 = \sum_{i=1}^d x_i^2$, the vector magnitude is expressed as a function of the activation variance:

$$\|h_n\| = \sqrt{d \cdot \sigma^2} \quad (18)$$

Upon substitution of this relation into the vocabulary-wide Softmax function, $\|h_n\|$ is mapped to the inverse temperature parameter $\beta = \frac{1}{T}$ from statistical mechanics [9, 10], a formulation closely related to post-hoc temperature scaling methods used in neural network calibration [21]:

$$P(w \mid C) = \frac{\exp(\beta \cdot \|u_w\| \cos(\theta_w))}{\sum_{v \in V} \exp(\beta \cdot \|u_v\| \cos(\theta_v))}, \quad \text{where } \beta = \|h_n\| \quad (19)$$

1. **High contextual certainty:** In predictable contexts, aligned information is written into the residual stream by successive attention heads and FFN layers. The variance across components σ^2 is inflated via constructive interference, causing $\|h_n\|$ to expand. A high inverse temperature β is yielded, thereby sharpening the distribution and lowering entropy.
2. **High contextual ambiguity:** In entropic or transitional contexts, orthogonal or destructive vectors are written into the residual stream by architectural sub-layers. The variance across components σ^2 is collapsed, shrinking $\|h_n\|$. A low inverse temperature β is yielded, flattening the distribution into a high-entropy state.

This behavior can be understood by treating the vector norm $\|h_n\|$ as a measure of contextual certainty. Under highly predictable contexts, the representation updates computed by successive attention and feed-forward sub-layers align along similar directions in the embedding space. This alignment increases the variance across the vector components, thereby expanding the magnitude $\|h_n\|$. In the thermodynamic analogy, a high inverse temperature β corresponds to a low-temperature state where the probability mass concentrates tightly on a single dominant token. Conversely, under high contextual ambiguity, updates from different sub-layers project along conflicting or orthogonal directions. The resulting destructive interference collapses the cross-component variance, shrinking the vector norm $\|h_n\|$. The corresponding low inverse temperature β yields a high-entropy probability distribution where probability mass is distributed more uniformly across candidate tokens.

5.2 Cosine similarity as a correlation coefficient

Similarly, by treating the dimensions of h_n and u_w as zero-mean random variables, the likelihood component $\cos(\theta_w)$ is mapped directly to the Pearson correlation coefficient (r_{h_n, u_w}) in feature space:

$$\cos(\theta_w) = \frac{\text{Cov}(h_n, u_w)}{\sigma_{h_n} \sigma_{u_w}} = r_{h_n, u_w} \quad (20)$$

where σ_{h_n} and σ_{u_w} denote the standard deviations of the components of h_n and u_w , respectively. The unnormalized logit is therefore proportional to the total feature covariance scaled by the model dimension: $z_w = \text{Cov}(h_n, u_w) \cdot d$. From an intuitive perspective, each component in the hidden space can be interpreted as representing a specific linguistic feature (e.g., tense, gender, or subject area). Under this interpretation, the context vector h_n and the token vector u_w serve as profiles of these features, and the cosine similarity measures the correlation between their respective configurations, a metric widely used to evaluate semantic similarity in distributed word representation spaces [22]. If the context vector exhibits large magnitude values in components corresponding to features such as “computing” and “plural”, and the token “servers” shares this pattern, their feature profiles will be highly correlated ($\cos(\theta_w) \approx 1$), yielding a high-likelihood match.

6 Statistical description of pre-training

While unembedding vectors u_w remain static during inference, both the context state h_n and the vocabulary matrix U are optimized simultaneously during pre-training via minimization of the cross-entropy loss [11] for a target next-token w^* :

$$\mathcal{L} = -\log \frac{\exp(h_n \cdot u_{w^*})}{\sum_{v \in V} \exp(h_n \cdot u_v)} \quad (21)$$

This optimization framework is modeled as a dual-force geometric process. Pre-training is conceptualized as a physical process of self-organization, where forces of attraction and repulsion shape the geometry of the embedding space, drawing a direct parallel to contrastive representation learning frameworks that rely on attractive and repulsive forces to construct coherent embedding manifolds [23]. The training loss functions as a potential energy surface, and gradient descent acts as a system of forces that pulls related vectors together while pushing unrelated ones apart:

6.1 The mutual attraction force

The gradient with respect to the correct token vector u_{w^*} is interpreted as an attractive force:

$$\frac{\partial \mathcal{L}}{\partial u_{w^*}} = -(1 - P(w^* | C))h_n \quad (22)$$

By this force, u_{w^*} and the context representation h_n are dynamically rotated toward one another, thereby maximizing the directional correlation $\cos(\theta_{w^*})$. Simultaneously, for a token w^* consistently validated across diverse contexts, the vector norm $\|u_{w^*}\|$ is driven upward to minimize corpus-wide loss, thereby establishing the global prior $P(w)$. The attraction force operates analogously to a spring pulling the correct token vector u_{w^*} and the context vector h_n together. The magnitude of this pull is inversely proportional to the model’s prediction confidence ($P(w^* | C) \rightarrow 0$). Over the course of training, this force rotates the context and token vectors toward alignment. Consequently, words that frequently appear as correct next-tokens are repeatedly pulled by diverse contexts, causing their vector norms to grow and establishing their high prior probability.

6.2 The global repulsion force

Conversely, a contrastive repulsion force against all incorrect tokens $v \neq w^*$ is generated by the denominator of the Softmax, resembling contrastive representation learning frameworks [12] and noise-contrastive estimation objectives [18]:

$$\frac{\partial \mathcal{L}}{\partial u_v} = P(v | C)h_n \quad (23)$$

The context state h_n is thereby pushed away from incorrect vocabulary vectors, forcing them toward orthogonality ($\cos(\theta_v) \rightarrow 0$). This repulsive force is scaled by the probability mass mistakenly assigned to each incorrect token, driving incorrect tokens to be orthogonal to the context vector. This projection clears the surrounding vector space to isolate and highlight the correct token representation.

6.3 Emergence of distributional semantics

Through loss minimization over a large corpus, semantic clustering emerges. For distinct tokens w_1 and w_2 sharing high contextual substitutability, identical directional updates are repeatedly received from shared context states h_n . A geometric steady-state is thereby established by this self-organizing system, wherein a coordinate system is discovered by vocabulary vectors such that spatial proximity mirrors latent semantic relationships, validating the distributional hypothesis [13]:

$$\cos(\theta_{w_1, w_2}) \rightarrow 1 \iff \mathcal{C}_{w_1} \approx \mathcal{C}_{w_2} \quad (24)$$

where $\mathcal{C}_w$ represents the distribution of contexts in which token w appears. For example, if the tokens “cat” and “dog” frequently appear in similar context sequences (e.g., “I fed the ___”), their respective unembedding vectors are updated along similar trajectories by the shared context vectors h_n . Even in the absence of explicit comparison during training, the shared contexts act as common basins of attraction that pull both vectors into spatial proximity. The resulting clustering behavior is an emergent property of the dual forces of attraction and repulsion, rather than a pre-programmed structural constraint.

7 Conclusion

By formalizing the large language model as a Bayesian engine, a transition is made from a qualitative understanding of neural network behaviors to an exact geometric framework. A context vector is iteratively constructed in tandem by multi-head attention and feed-forward

networks, where the final magnitude represents the statistical certainty (β) of the model, and the orientation represents structural feature configurations. The final token prediction is therefore represented as a direct measure of the Pearson correlation between this dynamic contextual variance and the self-organized coordinate landscape of the vocabulary embeddings.

References

- [1] Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L. and Polosukhin, I. (2017). Attention is all you need. *Advances in neural information processing systems*, 30.
- [2] Bengio, Y., Ducharme, R., Vincent, P. and Janvin, C. (2003). A neural probabilistic language model. *Journal of Machine Learning Research*, 3, pp.1137–1155.
- [3] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D. and Sutskever, I. (2019). Language models are unsupervised multitask learners. *OpenAI blog*, 1(9), p.9.
- [4] Press, O. and Wolf, L. (2017). Using the output embedding to improve language models. *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, pp.157–163.
- [5] Geva, M., Schuster, R., Berant, J. and Levy, O. (2021). Transformer feed-forward layers are key-value memories. *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pp.5484–5495.
- [6] Hendrycks, D. and Gimpel, K. (2016). Gaussian error linear units (GELUs). *arXiv preprint arXiv:1606.08415*.
- [7] Gao, J., He, D., Tan, X., Qin, T., Wang, L. and Liu, T.Y. (2019). Representation degeneracy problem in training natural language generation models. *International Conference on Learning Representations (ICLR)*.
- [8] Ba, J.L., Kiros, J.R. and Hinton, G.E. (2016). Layer normalization. *arXiv preprint arXiv:1607.06450*.
- [9] Jaynes, E.T. (1957). Information theory and statistical mechanics. *Physical review*, 106(4), p.620.
- [10] Ackley, D.H., Hinton, G.E. and Sejnowski, T.J. (1985). A learning algorithm for Boltzmann machines. *Cognitive Science*, 9(1), pp.147–169.
- [11] Shannon, C.E. (1948). A mathematical theory of communication. *The Bell System Technical Journal*, 27(3), pp.379–423.
- [12] van den Oord, A., Li, Y. and Vinyals, O. (2018). Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748*.
- [13] Harris, Z.S. (1954). Distributional structure. *Word*, 10(2-3), pp.146–162.
- [14] Luong, M.T., Pham, H. and Manning, C.D. (2015). Effective approaches to attention-based neural machine translation. *Proceedings of the 2015 Conference on Empirical Methods in Natural Language Processing*, pp. 1412–1421.
- [15] Miller, A., Fisch, A., Dodge, J., Karimi, A.H., Bordes, A. and Weston, J. (2016). Key-value memory networks for question answering. *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*, pp. 440–450.

- [16] Elhage, N., Nanda, N., Olsson, C., Henighan, T., Joseph, N., Mann, B., Askell, A., Yang, Y., DasSarma, N., Jones, A. and others (2021). A mathematical framework for transformer circuits. *Transformer Circuits Thread*.
- [17] nostalgebraist (2020). interpreting GPT: the logit lens. *LessWrong*.
- [18] Gutmann, M. and Hyvärinen, A. (2010). Noise-contrastive estimation: A new estimation principle for unnormalized statistical models. *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pp. 297–304.
- [19] Mikolov, T., Karafiát, M., Burget, L., Cernocký, J. and Khudanpur, S. (2010). Recurrent neural network based language model. *Interspeech*, 2, pp. 1045–1048.
- [20] Sukhbaatar, S., Szlam, A., Weston, J. and Fergus, R. (2015). End-to-end memory networks. *Advances in neural information processing systems*, 28.
- [21] Guo, C., Pleiss, G., Sun, Y. and Weinberger, K.Q. (2017). On calibration of modern neural networks. *International Conference on Machine Learning*, pp. 1321–1330.
- [22] Mikolov, T., Sutskever, I., Chen, K., Corrado, G.S. and Dean, J. (2013). Distributed representations of words and phrases and their compositionality. *Advances in neural information processing systems*, 26.
- [23] Hadsell, R., Chopra, S. and LeCun, Y. (2006). Dimensionality reduction by learning an invariant mapping. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, 2, pp. 1735–1742.